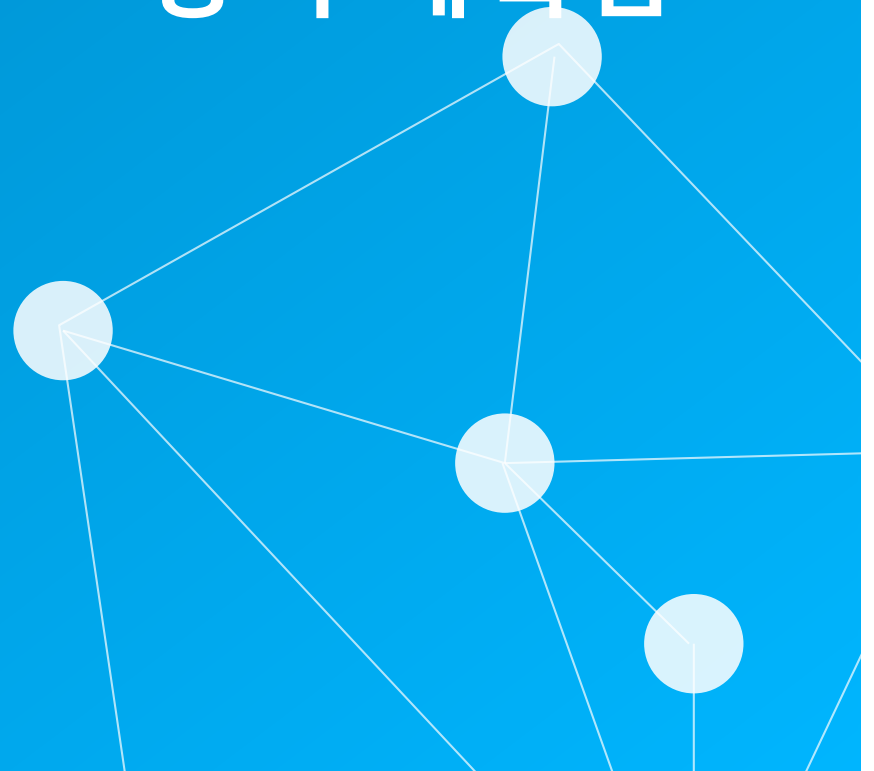


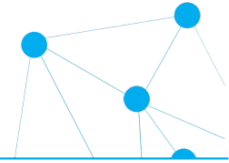
07

CHAPTER

동적 계획법



학습 내용



7.1 피보나치수열과 동적 계획법

7.2 이항계수 구하기

7.3 배낭 채우기 문제: 0-1 Knapsack

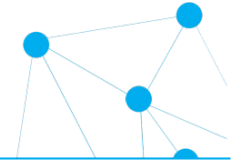
7.4 최장 공통 부분순서 문제

7.5 그래프의 인접 행렬 표현과 최단 경로 문제

7.6 모든 정점간의 최단 경로 길이

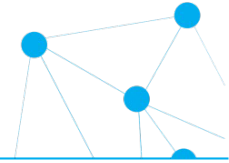
7.7 편집 거리

동적 계획법



- Dynamic programming
 - 1950년대에 미국의 수학자인 벨만(Richard Bellman)
 - 다단계의 의사 결정 프로세스를 최적화하는 일반적인 방법
 - → 동적 계획법으로 번역
- 동적 계획법
 - 정복 기법과 유사한 부분이 많음
 - 부분 문제들의 답을 저장, 다시 꺼내서 사용
 - 같은 부분 문제를 다시 풀지 않도록 하는 것이 핵심
 - 공간으로 시간을 버는 전략의 일종

7.1 피보나치수열과 동적 계획법



- 피보나치 수열 (분할 정복)

$$fib(n) = \begin{cases} n & n \leq 1 \\ fib(n-2) + fib(n-1) & \text{otherwise} \end{cases}$$

– 심각한 계산의 중복: $\rightarrow O(2^n)$

- 동적 계획법

– 어렵게 구한 답을 한 번만 쓰고 버리지 말고 저장 $\rightarrow$ 재사용

– 구현 방법

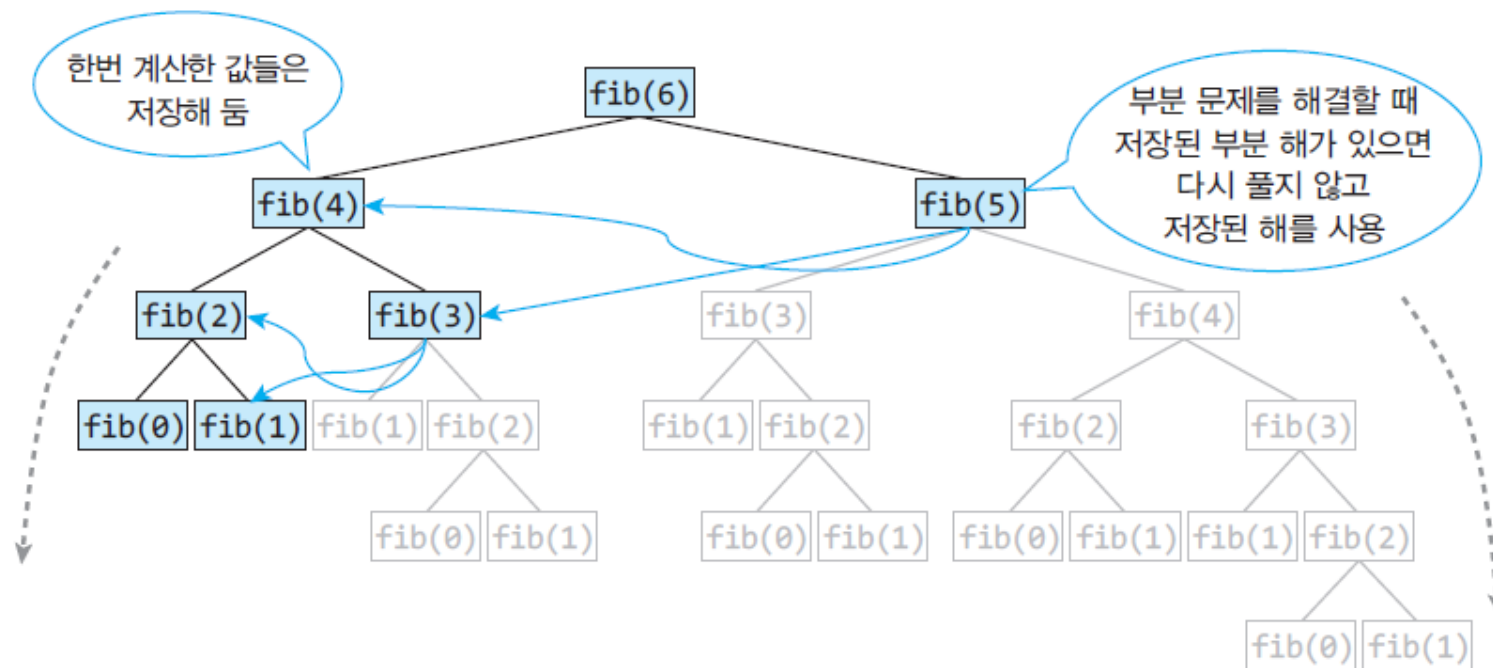
- 메모이제이션(memoization): 하향식(top-down)

- 테이블화(tabulation): 상향식(bottom-up)

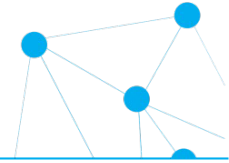
하향식
(메모이제이션)
상향식
(테이블화)

메모이제이션을 이용한 피보나치수열

문제를 풀 때마다 이미 풀린 문제인지를 먼저 확인한다. 풀린 문제이면 저장된 답을 이용하고, 풀리지 않은 문제이면 그 문제를 풀고 답을 다시 저장한다.



알고리즘과 복잡도



알고리즘 7.1 피보나치수열(메모이제이션 이용)

```
01 def fib_dp_mem(n) :  
02     if( mem[n] == None ) :           # 풀리지 않은 경우 → 계산하고 저장  
03         if n < 2 :  
04             mem[n] = n               # 기반 상황: n≤1  
05         else:                       # 일반 상황: otherwise  
06             mem[n] = fib_dp_mem(n-1) + fib_dp_mem(n-2)  
07     return mem[n]
```

- 시간 복잡도
 - $O(n)$
- 공간 복잡도
 - $O(n)$

def fibo(n):
 if (f[n] == None):
 f[n] = fibo(n-1) + fibo(n-2)
 return f[n]

while

테이블화를 이용한 피보나치수열

결과를 저장할 테이블을 먼저 만든다. 다음으로 답이 이미 알려진 단순한 상황, 즉 기반 상황 (base case)에 대한 테이블 항목을 먼저 채우고, 이들을 바탕으로 테이블을 채워서 올라간다.

피보나치수열의 초기 조건은 $\text{fib}(0)=0$ 과 $\text{fib}(1)=1$ 이지. 이것은 기반 상황들인데, 확실한 답이니까 가장 먼저 저장해 둬야 해.



fib(6)	
fib(5)	
fib(4)	
fib(3)	2
fib(2)	1
fib(1)	1
fib(0)	0

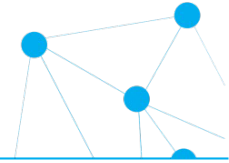


$\text{fib}(3)$ 는 $\text{fib}(1)$ 과 $\text{fib}(2)$ 를 표에서 찾아서 계산하면 $1+1=2$. 결과를 다시 저장.



$\text{fib}(2)$ 는 $\text{fib}(0)$ 과 $\text{fib}(1)$ 을 더하면 되니까... 표에서 찾아서 계산하면 $1+0=1$. 결과를 다시 표에 저장.

알고리즘과 복잡도



알고리즘 7.2 피보나치수열(테이블화)

```
01 def fib_dp_tab(n) :  
02     f = [None] * (n+1)  
03     f[0] = 0  
04     f[1] = 1  
05     for i in range(2, n + 1):  
06         f[i] = f[i-1] + f[i-2]  
07     return f[n]
```

- 시간 복잡도
 - $O(n)$
- 공간 복잡도
 - $O(n)$

알고리즘 테스트

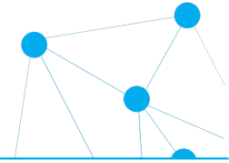
테이블화와 메모이제이션을 이용한 피보나치수열

```
n = 8  
print('동적계획( 테이블화 ): Fibonacci(%d) = %n, fib_dp_tab(n))  
mem = [None] * (n+1)  
print('동적계획(메모이제이션): Fibonacci(%d) = %n, fib_dp_mem(n))
```

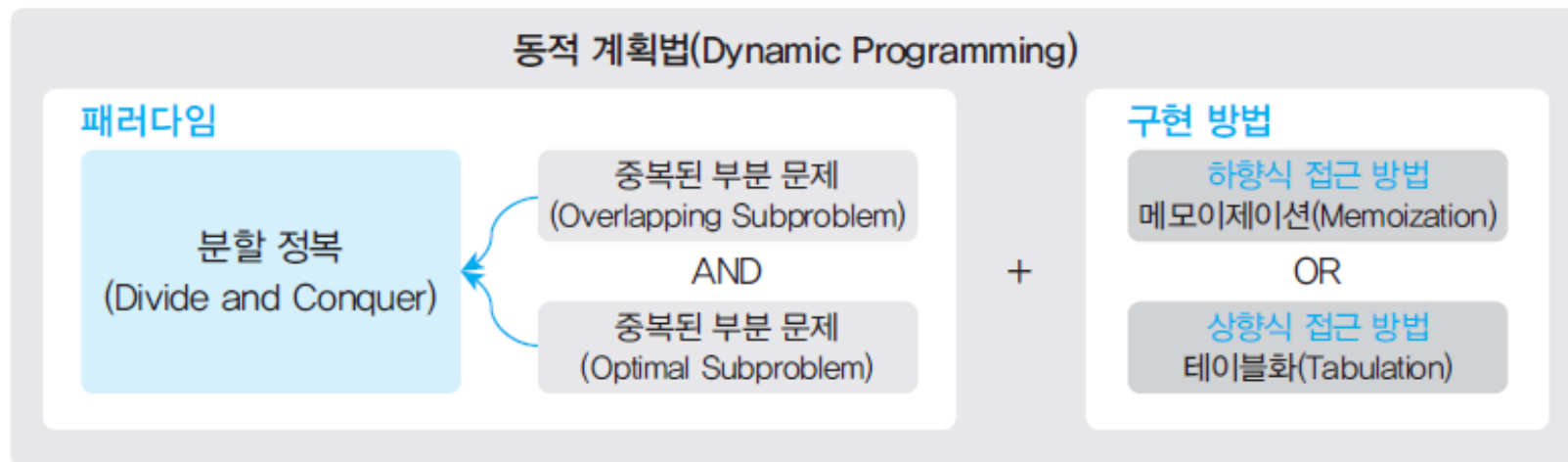
C:\WINDOWS\system32\cmd.exe

```
동적계획( 테이블화 ): Fibonacci(8) = 21  
동적계획(메모이제이션): Fibonacci(8) = 21
```

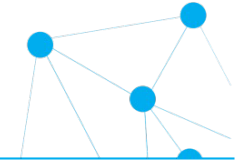
동적 계획법 패러다임



- 문제의 특성
 - (1) 겹치는 부분 문제(overlapping subproblem)
 - (2) 최적 부분 구조(optimal substructure)
 - 부분 문제의 최적해들을 이용하면 전체문제의 최적해를 구할 수 있는 구조
- 패러다임과 구현 방법



7.2 이항계수 구하기

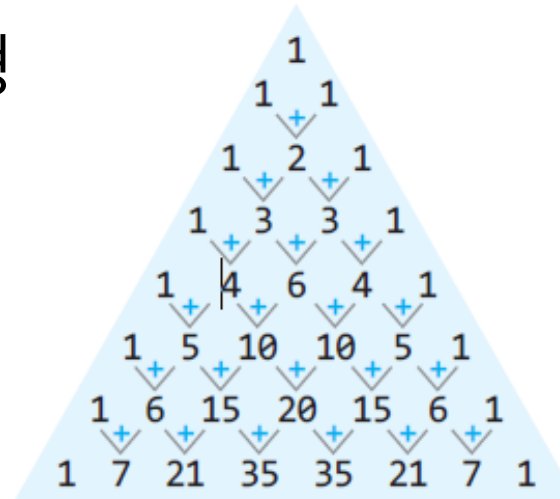


- 이항정리와 파스칼의 삼각형
 - 이항 정리

$$(a+b)^n = \sum_{k=0}^n C(n, k) a^{n-k} b^k$$

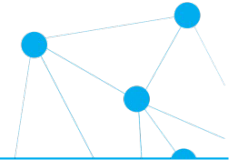
- 이항 계수

$$C(n, k) = \frac{n!}{k!(n-k)!}$$



➔ 동적 계획법에 의한 풀이

이항계수의 순환 관계식



- $C(n, k)$: n 개에서 k 개를 뽑는 순서 없는 조합의 가짓수

- 순환 관계식 만들기

- 기반 상황

- 답이 이미 알려진 가장 작은 부분 문제

- $C(n, 0) = 1$

- $C(n, n) = 1$

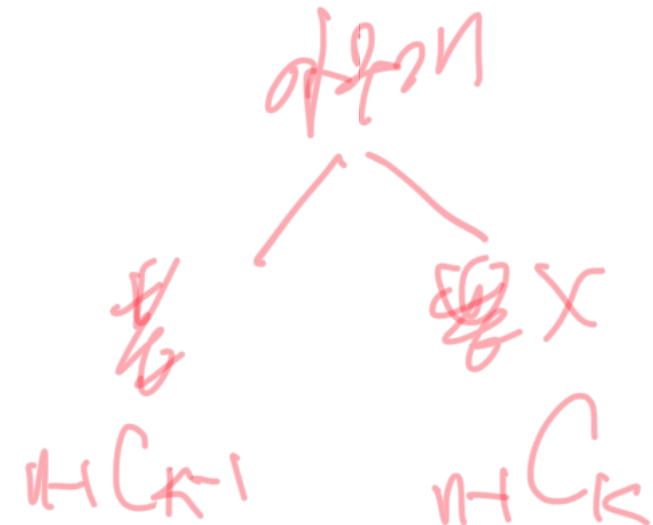
- 일반 상황

- n 번째 항목을 포함하는 경우: $C(n-1, k-1)$

- n 번째 항목을 포함하지 않는 경우: $C(n-1, k)$

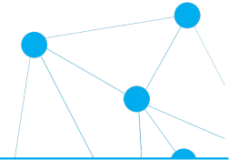
- 순환 관계식

$$C(n, k) = \begin{cases} 1 & k = 0, k = n \\ C(n-1, k-1) + C(n-1, k) & \text{otherwise} \end{cases}$$



포함 포함 X

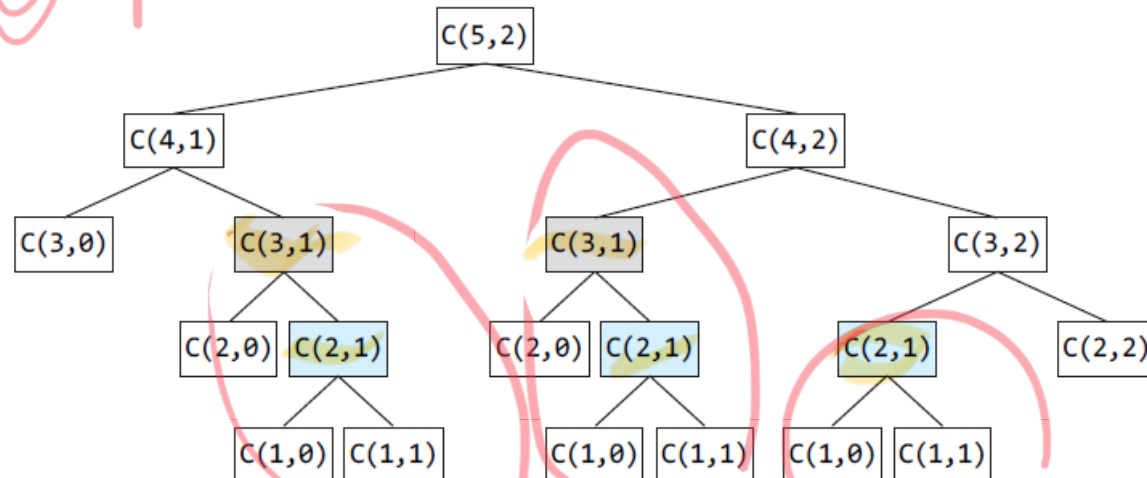
분할 정복에 의한 이항계수



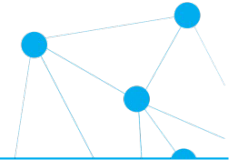
알고리즘 7.3 이항계수 $C(n, k)$ 계산 함수(분할 정복 기법)

```
01 def bino_coef_dc(n , k):  
02     if k==0 or k ==n :  
03         return 1  
04     return bino_coef_dc(n-1 , k-1) + bino_coef_dc(n-1 , k)
```

- 많은 중복이 발생



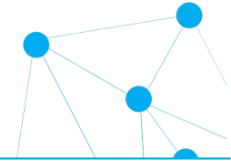
동적 계획법에 의한 이항계수



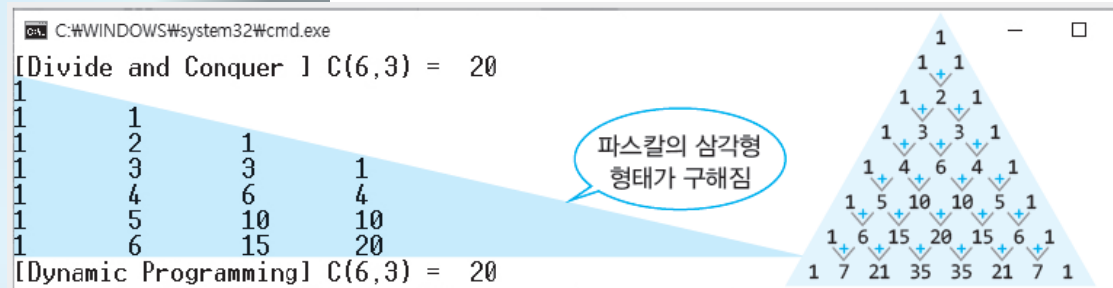
- 테이블 $\rightarrow$ 공복해결을 위해 사용
 - 두 개의 변수 $\rightarrow$ 2차원 테이블
 - 예: $C(5,3)$

	$k=0$	$k=1$	$k=2$	$k=3$
$n=0$	1			
$n=1$	1	1		
$n=2$	1	2	1	
$n=3$	1	3	3	1
$n=4$	1	4	6	4
$n=5$	1	5	10	10

알고리즘



테이블 기반
재귀



- 시간 복잡도: $O(nk)$
- 공간 복잡도: $O(nk)$

이항 계수 by Top-Down DP

bc[][] : 결과를 담는 테이블

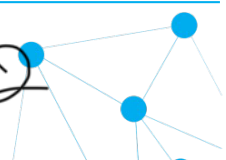
```
long long combi(int n, int r)
{
    long long a, b;
    if(r > n) return 0;
    if(bc[n][r] > 0) return bc[n][r];
    if(n == r | r == 0) {
        bc[n][r] = 1;
        return 1;
    }
    a = combi(n-1, r);
    b = combi(n-1, r-1);
    bc[n][r] = a+b;

    return bc[n][r];
}
```

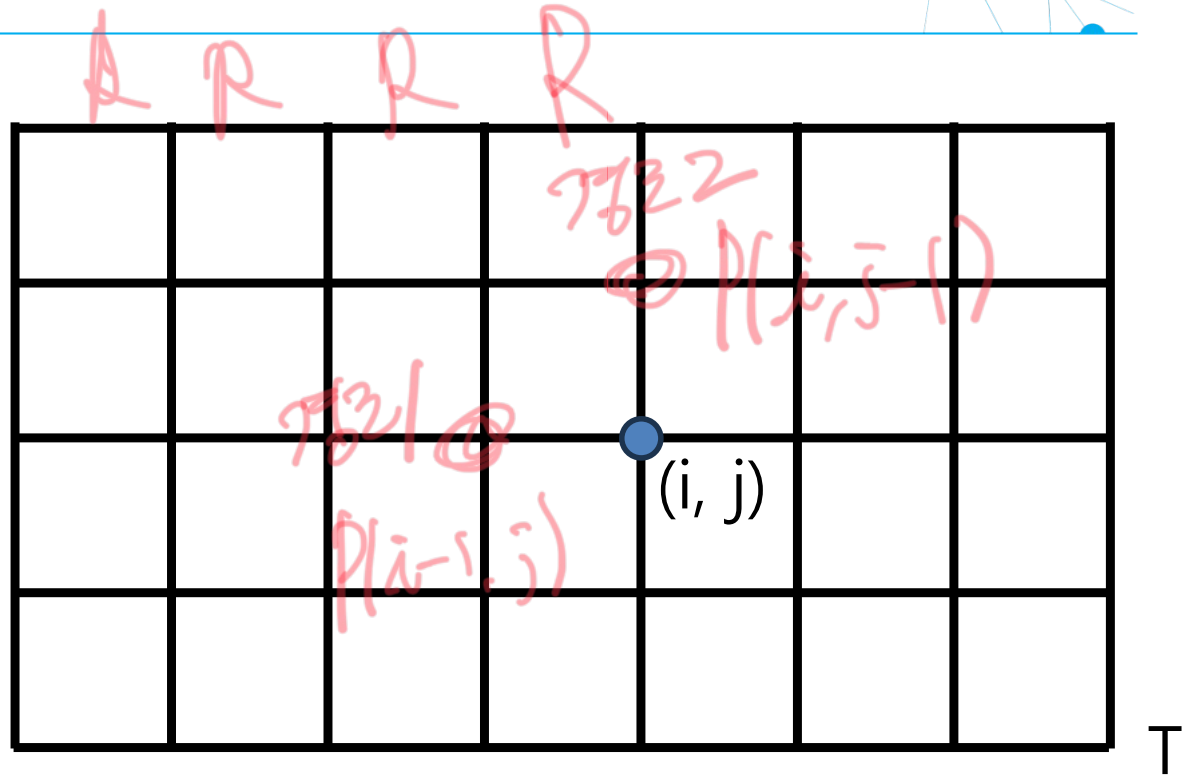
(- DP 7(8))

+ 메모리 최적화 (8)

격자 경로 1

$$\rightarrow |2022| = 6C2$$


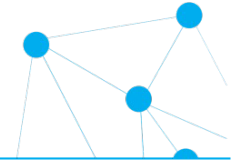
- S에서 T 까지의 서로 다른 최단경로의 개수는?
- '경우의 수' 계산 하듯이 할 수도 있지만...



$$P(i, j) = \begin{cases} 1, & i = 1 \text{ or } j = 1 \\ P(i-1, j) + P(i, j-1), & \text{otherwise} \end{cases}$$

경로 + 경로

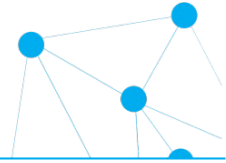
격자 경로 1



S	1	1	1	1	1	1	1
1	2	3	4	5	6	7	8
1	3	6	10	15	21	28	36
⌈							
⌈							
							T

$$P(i, j) = \begin{cases} 1, & i = 1 \text{ or } j = 1 \\ P(i - 1, j) + P(i, j - 1), & \text{otherwise} \end{cases}$$

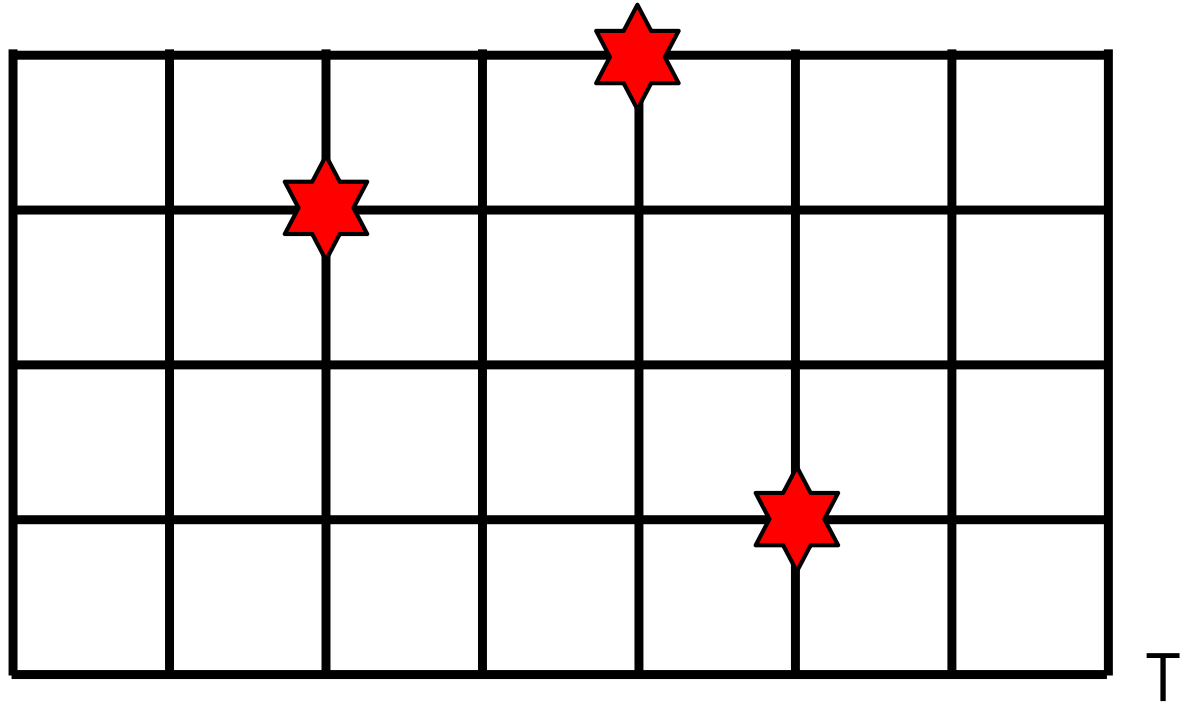
격자 경로 2



- S에서 T까지의 서로 다른 최단경로의 개수는?

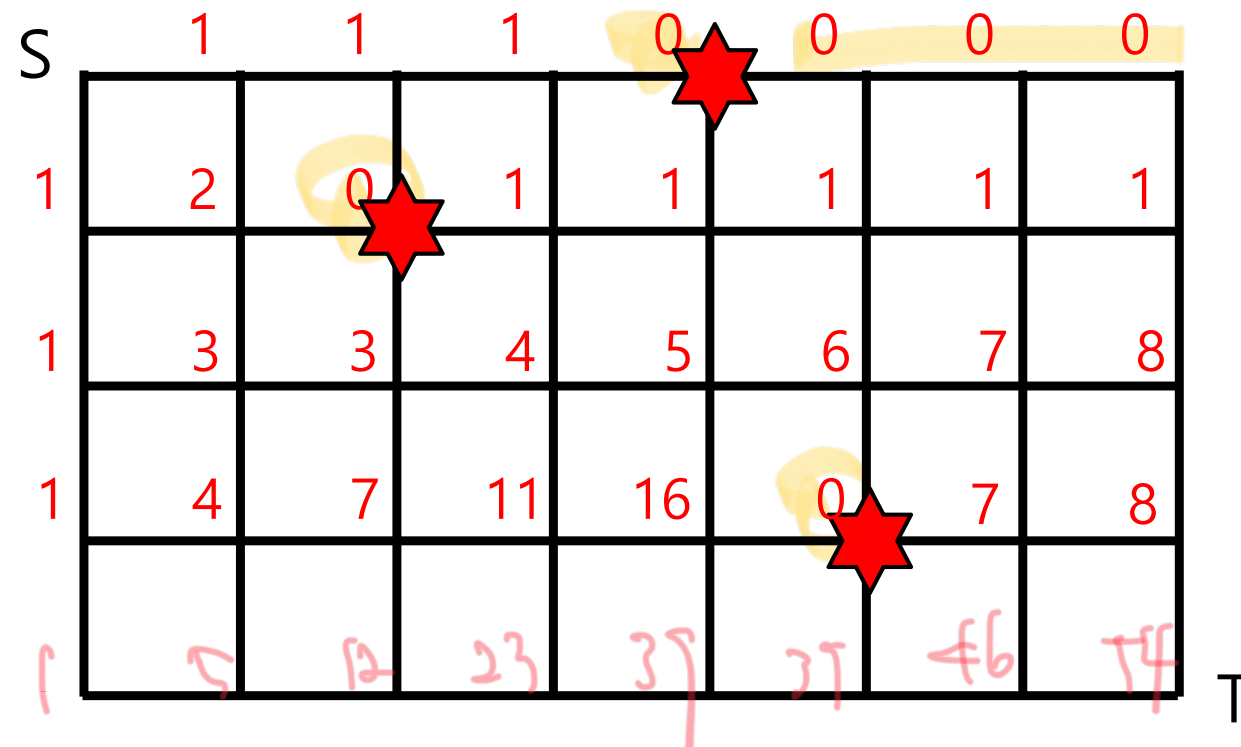
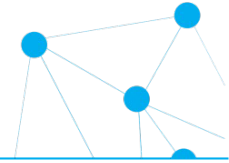
- 단, ★ 표시된 곳은 지날 수 없다.

- '경우의 수' 계산 하듯이 하기는 쉽지 않다



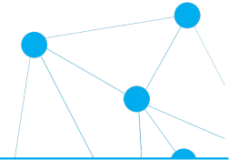
$$P(i, j) = \begin{cases} 0, & \text{if marked} \\ 1, & i = 1 \text{ or } j = 1 \\ P(i - 1, j) + P(i, j - 1), & \text{if not marked} \end{cases}$$

격자 경로 2

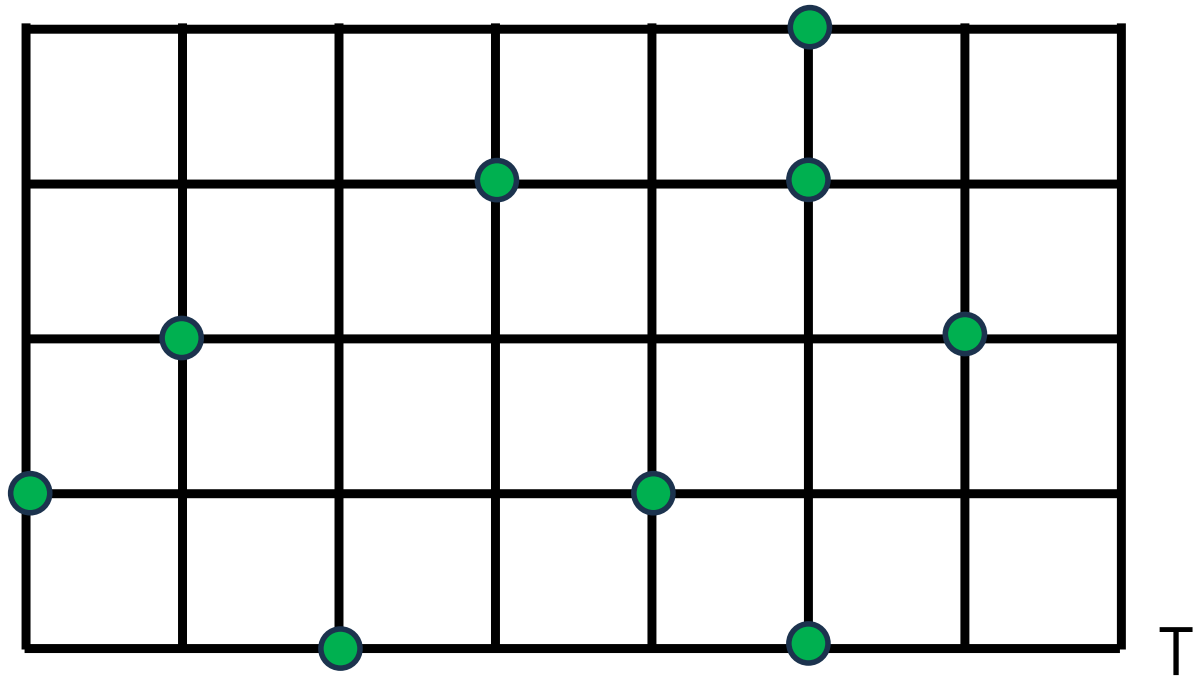


$$P(i, j) = \begin{cases} 0, & \text{if marked} \\ 1, & i = 1 \text{ or } j = 1 \\ P(i - 1, j) + P(i, j - 1), & \text{if not marked} \end{cases}$$

격자 경로 3

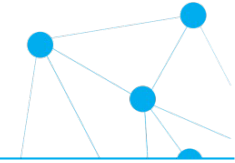


- S에서 T까지의 서로 다른 최단경로의 개수는?
- 단, ● 표시된 곳을 최소 k번 지나야 한다.

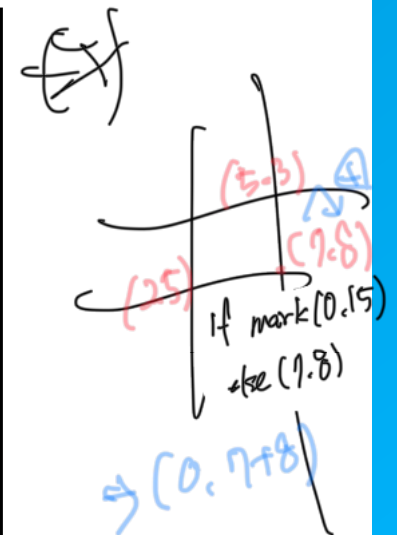
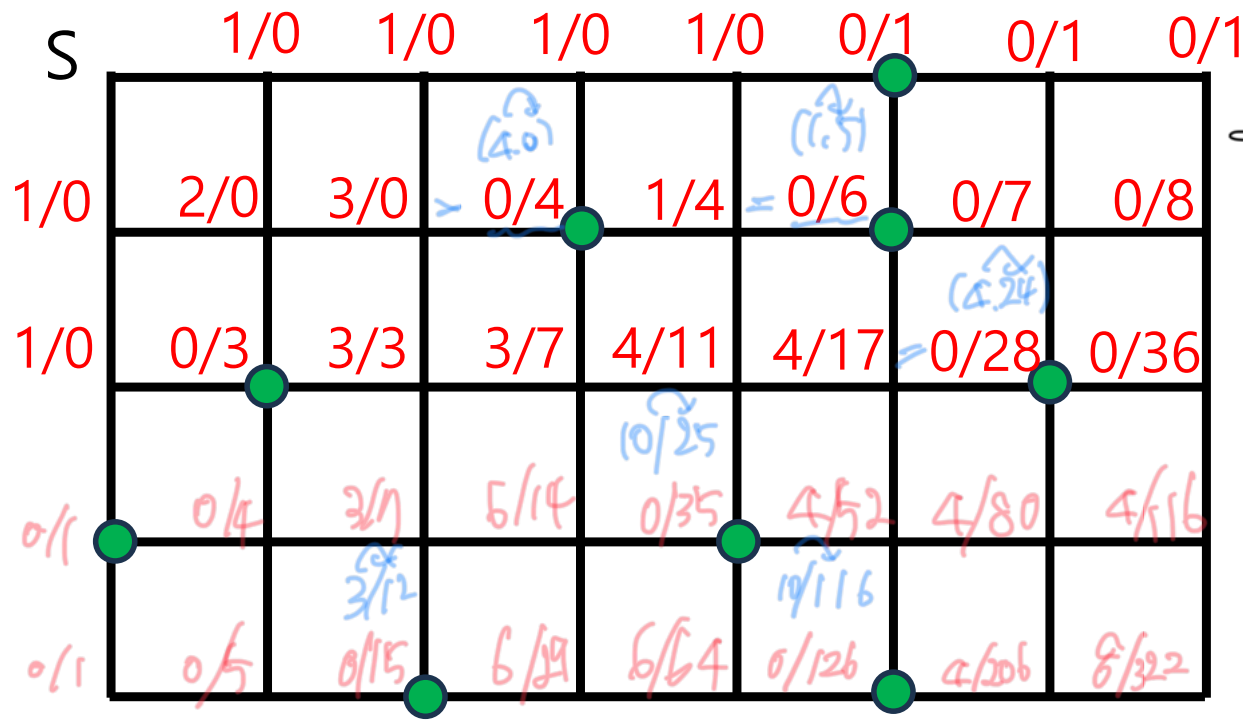


대안: 2는 ★로 생각하고 여집합하면 됨.

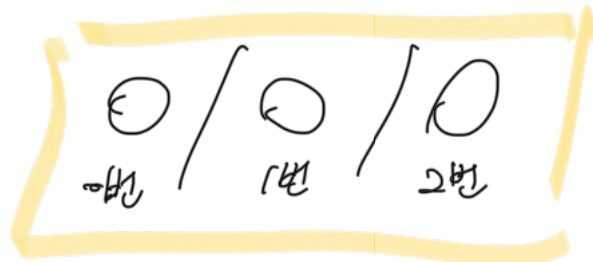
격자 경로 3



- 재귀식을 만들기 전에 예부터 보자...
- $k=1$ 로 두자. 즉, 표시된 곳을 한 번 이상 지나는 경로의 수를 구하자.
- 표시된 곳을 지난 경우와 지나지 않은 경로를 구분하여 계산

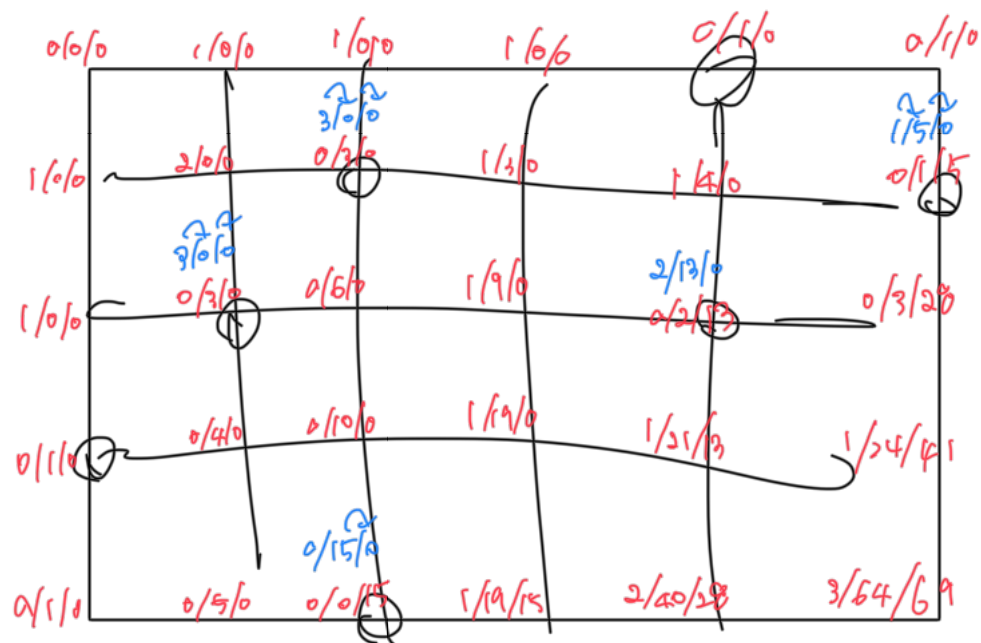
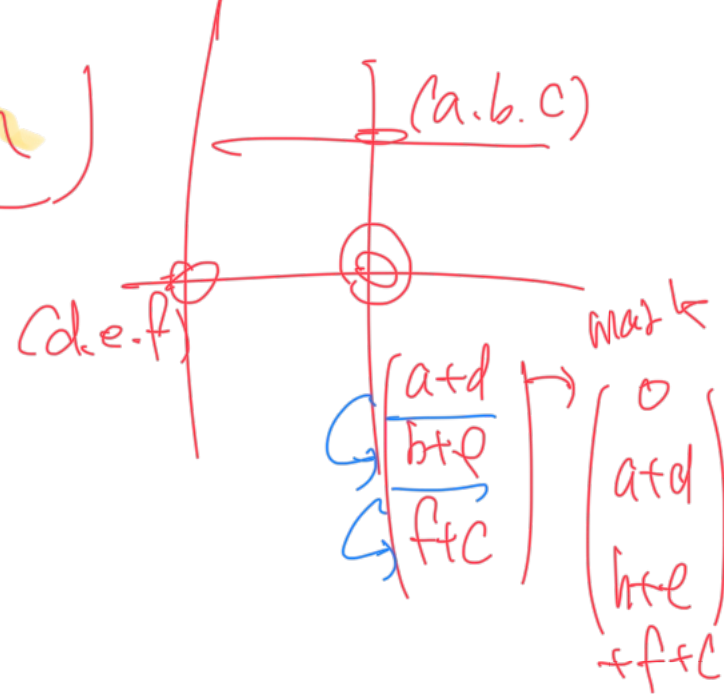


$k=2$ 일 때

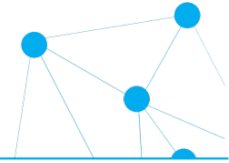


Solution

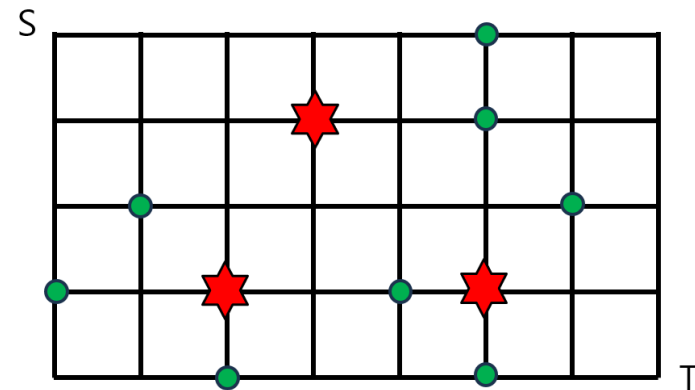
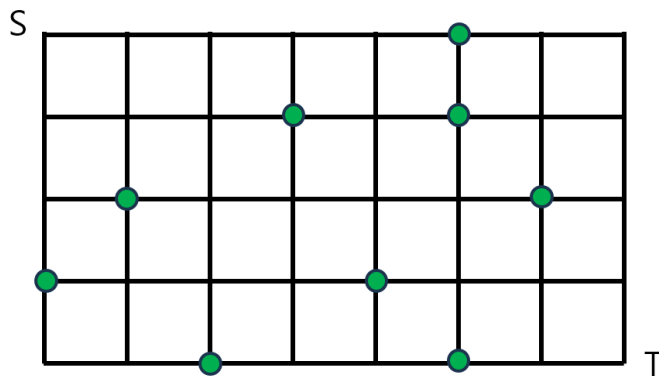
$k=3$ 일 때 $= 1/0/0/0 \dots$



격자 경로 3



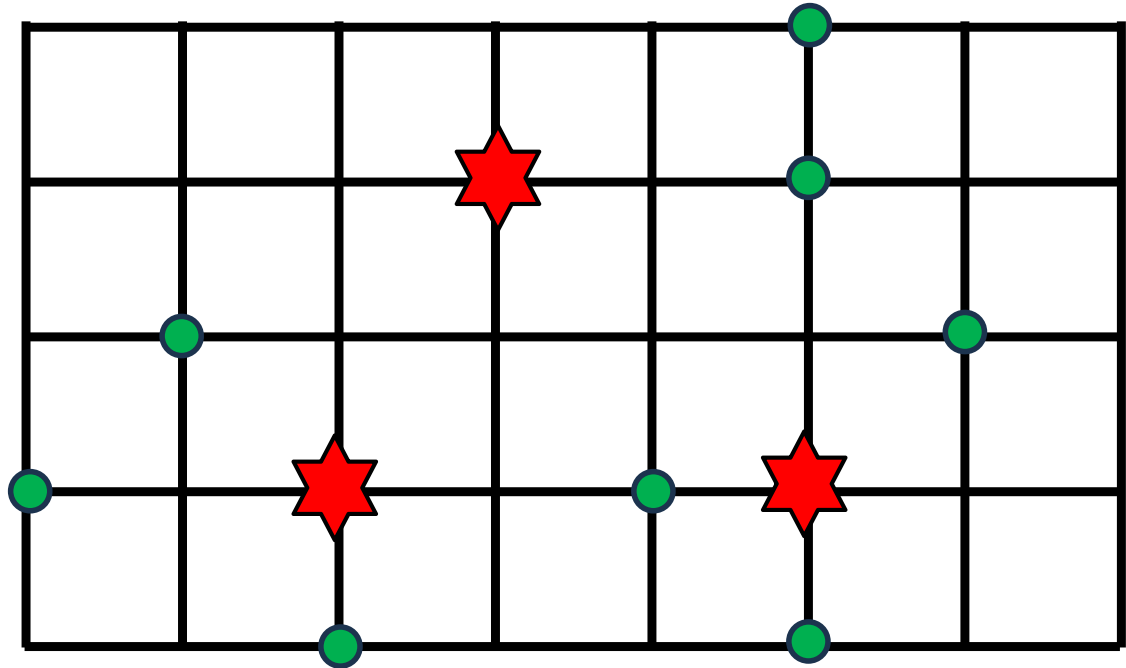
- 표시된 곳을 **k번 이상** 지나는 경로 수는?
- 표시된 곳을 **정확히 k번** 지나는 경로 수는?
- 지나갈 수 없는 지점도 있고 표시된 곳을 k번 (이상) 지나가야 하는 지점이 혼재된 경우, 경로 수는?



격자 경로 3

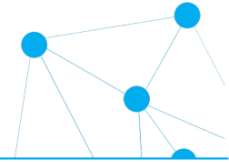
행 ,

S



T

Coin Changing (일명 거스름 돈 문제)



- 액면가가 서로 다른 동전 종류가 n 개 있고, 각 종류마다 동전의 개수는 충분히 있다.
- 단, 최소 액면가의 동전은 1원이다.
- 동전으로 거스름 돈을 지불할 때 최소 개수의 동전으로 지불한다.
- 그 때 필요한 동전의 개수는?

Coin Changing (일명 거스름 돈 문제)

- Example

- 거스름 돈: 12

- 3 종류의 동전: $denom[1]=1$, $denom[2]=6$, $denom[3]=10$

		j												
		0	1	2	3	4	5	6	7	8	9	10	11	12
i	1	0	1	2	3	4	5	6	7	8	9	10	11	12
	2	0	1	2	3	4	5	1	2	3	4	5	6	2
	3	0	1	2	3	4	5	1	2	3	4	1	2	2

$$c[i][j] = \begin{cases} 0 \\ c[i-1][j] \\ \min\{c[i-1][j], 1 + c[i][j - denom[i]]\} \end{cases}$$

if $j = 0$

if $denom[i] > j$

if $denom[i] \leq j$

Coin Changing (일명 거스름 돈 문제)

- Another example

- 거스름 돈: 25

- 3 종류의 동전: $denom[1]=1$, $denom[2]=7$, $denom[3]=15$

		j																											
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25		
i	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25		
	2	0	1	2	3	4	5	6	1	2	3	4	5	6	7	2	3	4	5	6	7	8	3	4	5	6	7		
	3	0	1	2	3	4	5	6	1	2	3	4	5	6	7	2	1	2	3	4	5	6	3	2	3	4	5		

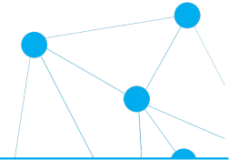
$$c[i][j] = \begin{cases} 0 \\ c[i-1][j] \\ \min\{c[i-1][j], 1 + c[i][j - denom[i]]\} \end{cases}$$

if $j = 0$

if $denom[i] > j$

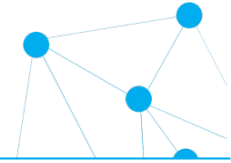
if $denom[i] \leq j$

동적 계획법



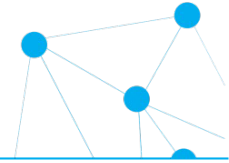
- 여기서부터는 <보충> 자료로 강의합니다.
- <보충> 자료가 끝난 후 다시 이곳으로 돌아와 계속 진행합니다.

7.3 배낭 채우기 문제: 0-1 Knapsack



- 억지기법: 완전 탐색(문제 3.7)
 - 원소의 개수가 n 인 집합의 부분집합의 수가 $2^n \rightarrow O(2^n)$
- 배낭 문제의 순환 관계식 만들기
 - 배낭 용량 W
 - n 개의 물건: $E_1, E_2, \dots, E_n$
 - $E_i = (wt_i, val_i)$
 - $A(n, W)$: 배낭의 최대 가치

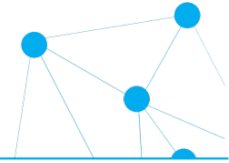
배낭 문제의 순환 관계식 만들기



- $A(k, w)$ 의 기반 상황
 - $A(0, w) = 0, A(k, 0) = 0$
- $A(k, w)$ 의 일반 상황
 - Case1: $wt_k > w \rightarrow$ 넣을 수 없음
 - $A(k, w) = A(k - 1, w)$
 - Case2: $wt_k \leq w \rightarrow$ 넣거나, 안 넣거나
 - 넣은 경우: $A(k - 1, w - wt_k) + val_k$
 - 넣지 않은 경우: $A(k - 1, w)$
- 순환 관계식

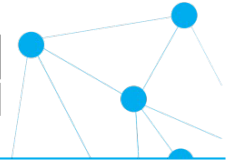
$$A(n, W) = \begin{cases} 0 & \text{if } W = 0 \text{ or } n = 0 \\ A(n-1, W) & \text{if } wt_n > W \\ \max(val_n + A(n-1, W - wt_n), A(n-1, W)) & \text{otherwise} \end{cases}$$

분할 정복 알고리즘



- 많은 중복 발생

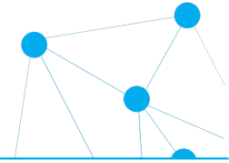
동적 계획법에 의한 0-1 배낭 문제 해법



- 문제에 대한 추가적인 제한
 - 무게의 단위를 정수로 제한 → 동적 계획법 적용 가능
- 테이블 설계
 - $(n + 1) * (W + 1)$

		배낭 용량						
		0	1	...	$w - wt_i$	w	...	W
물 건 의 수	0	0	0	...	0	0	...	0
	1	0						
	...	...						
	$i-1$	0			$A(i-1, w - wt_i)$	$A(i-1, w)$		
	i	0				$A(i, w)$		
	...	...						
	n	0						$A(n, W)$

알고리즘



알고리즘 테스트

0-1 배낭 채우기 문제에 대한 분할 정복과 동적 계획법

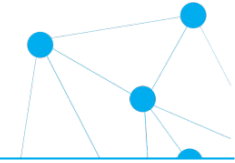
```
val = [60, 100, 190, 120, 200, 150]
wt = [2, 5, 8, 4, 7, 6]
W = 18
n = len(val)
print("0-1배낭문제(분할 정복): ", knapSack_bf(W, wt, val, n))
print("0-1배낭문제(동적 계획): ", knapSack_dp(W, wt, val, n))
```

C:\WINDOWS\system32\cmd.exe

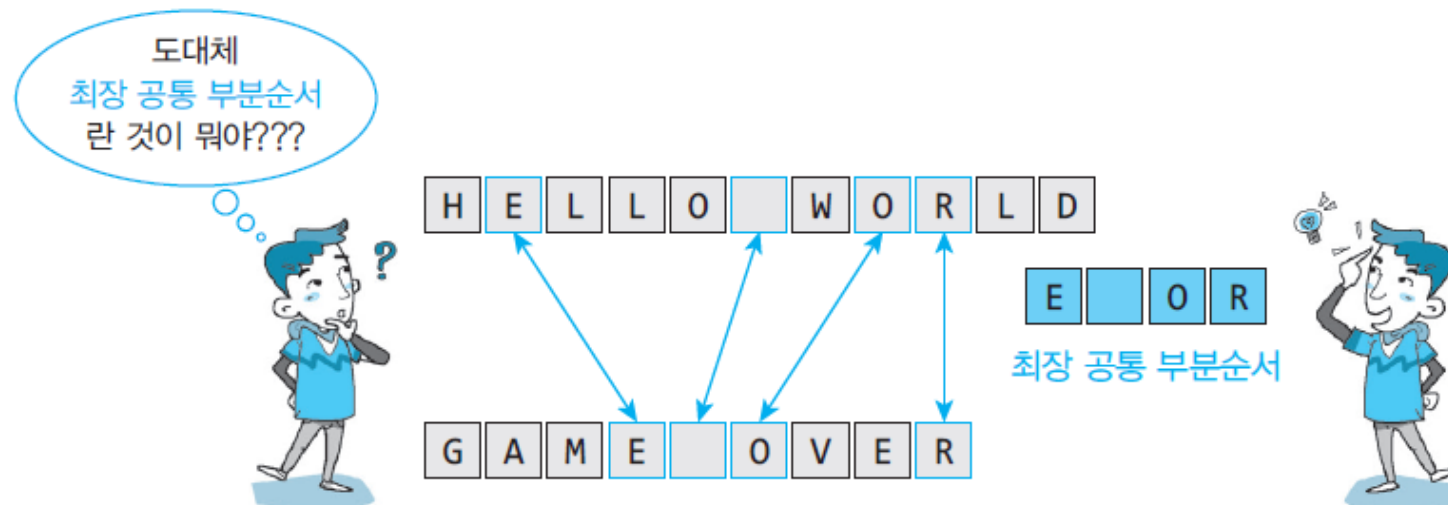
```
0-1배낭문제(분할 정복): 480
0-1배낭문제(동적 계획): 480
```

- 시간 복잡도: $O(nW)$
- 공간 복잡도: $O(nW)$

7.4 최장 공통 부분 순서 문제



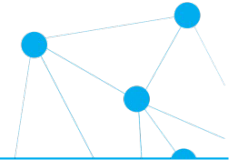
- Longest Common Subsequence, **LCS**
 - 데이터의 유사도를 평가
 - 유전자 염기 서열, 두 소스 파일의 차이 등



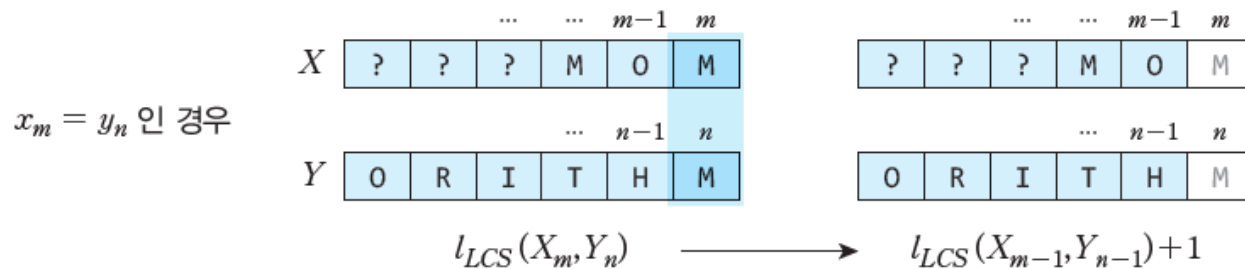
[그림 7.10] 최장 공통 부분순서 문제의 예

- LCS의 길이: $l_{lcs}(X_m, Y_n)$
 - $X = \langle x_1, x_2, \dots, x_m \rangle$, $Y = \langle y_1, y_2, \dots, y_n \rangle$

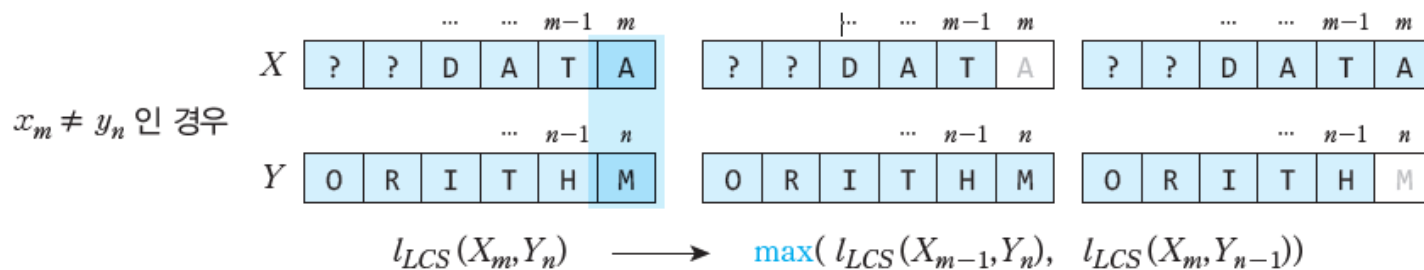
LCS 길이의 순환 관계식



- $l_{lcs}(X_m, Y_n)$ 의 기반 상황
 - $l_{lcs}(X_0, Y_n) = l_{lcs}(X_m, Y_0) = 0$
- $l_{lcs}(X_m, Y_n)$ 의 일반 상황
 - 마지막 문자를 중심으로 두 경우로 나눔

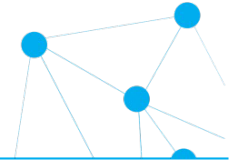


[그림 7.11] 마지막 문자가 같은 경우



[그림 7.12] 마지막 문자가 서로 다른 경우

순환구조 알고리즘



- 순환 관계식

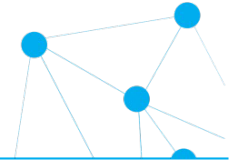
$$l_{LCS}(X_m, Y_n) = \begin{cases} 0 & m = 0 \text{ or } n = 0 \\ 1 + l_{LCS}(X_{m-1}, Y_{n-1}) & x_m = y_n \\ \max(l_{LCS}(X_{m-1}, Y_n), l_{LCS}(X_m, Y_{n-1})) & \text{otherwise} \end{cases}$$

- 순환구조 알고리즘

알고리즘 7.7 최장 공통 부분순서(순환 구조)

```
01 def lcs_recur(X, Y, m, n):
02     if m == 0 or n == 0:           # base case
03         return 0
04     elif X[m-1] == Y[n-1]:         # case 1: x_m == y_n
05         return 1 + lcs_recur(X, Y, m-1, n-1)
06     else:                          # case 2
07         return max(lcs_recur(X, Y, m, n-1), lcs_recur(X, Y, m-1, n))
```


동적 계획법에 의한 LCS 길이



- 테이블

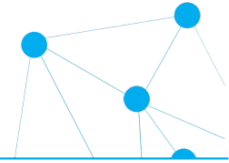
- $(m + 1) * (n + 1)$
- $L[0][n] = L[m][0] = 0$

$$L[m][n] = \begin{cases} 0 & m = 0 \text{ or } n = 0 \\ 1 + L[m-1][n-1] & x_m = y_n \\ \max(L[m-1][n], L[m][n-1]) & \text{otherwise} \end{cases}$$

		H	E	L	L	O	'	W	O	R	L	D
	0	0	0	0	0	0	0	0	0	0	0	0
G	0	0	0	0	0	0	0	0	0	0	0	0
A	0	0	0	0	0	0	0	0	0	0	0	0
M	0	0	0	0	0	0	0	0	0	0	0	0
E	0	0	1	1	1	1	1	1	1	1	1	1
'	0	0	1	1	1	1	2	2	2	2	2	2
O	0	0	1	1	1	2	2	2	3	3	3	3
V	0	0	1	1	1	2	2	2	3	3	3	3
E	0	0	1	1	1	2	2	2	3	3	3	3
R	0	0	1	1	1	2	2	2	3	4	4	4

- 시간 복잡도 = 공간 복잡도: $O(mn)$

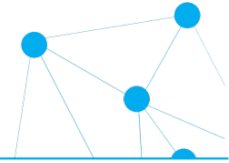
LCS 추적 알고리즘



- LCS 테이블에서 LCS 문자열을 추적하는 과정

		H	E	L	L	O	''	W	O	R	L	D
	0	0	0	0	0	0	0	0	0	0	0	0
G	0	↑0	0	0	0	0	0	0	0	0	0	0
A	0	↑0	0	0	0	0	0	0	0	0	0	0
M	0	↑0	0	0	0	0	0	0	0	0	0	0
E	0	0	↖1	←1	←1	←1	1	1	1	1	1	1
''	0	0	1	1	1	1	↖2	←2	2	2	2	2
O	0	0	1	1	1	2	2	2	↖3	3	3	3
V	0	0	1	1	1	2	2	2	↑3	3	3	3
E	0	0	1	1	1	2	2	2	↑3	3	3	3
R	0	0	1	1	1	2	2	2	3	↖4	←4	←4

알고리즘 테스트



알고리즘 테스트

LCS 길이 알고리즘 테스트

```
X = "GAME OVER"
Y = "HELLO WORLD"
print("X = ", X)
print("Y = ", Y)
print("LCS(분할 정복)", lcs_dc(X, Y, len(X), len(Y)))
print("LCS(동적 계획)", lcs_dp(X, Y) )
```

```
C:\WINDOWS\system32\cmd.exe
X = GAME OVER
Y = HELLO WORLD
LCS(분할 정복) 4
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
[0, 0, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2]
[0, 0, 1, 1, 1, 2, 2, 2, 3, 3, 3, 3]
[0, 0, 1, 1, 1, 2, 2, 2, 3, 3, 3, 3]
[0, 0, 1, 1, 1, 2, 2, 2, 3, 3, 3, 3]
[0, 0, 1, 1, 1, 2, 2, 2, 3, 4, 4, 4]
LCS = E OR
LCS(동적 계획) 4
```

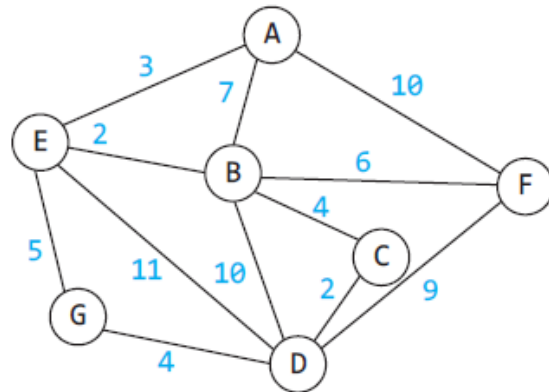
7.5 그래프 인접 행렬 표현과 최단 경로 문제

- 최단 경로(shortest path) 문제
 - 가중치 그래프에서 두 정점을 연결하는 여러 경로들 중 에서 간선들의 가중치 합이 최소가 되는 경로를 찾는 문제
 - 예: "최단시간 경로", "최단거리 경로" 등



인접 행렬을 이용한 가중치 그래프의 표현

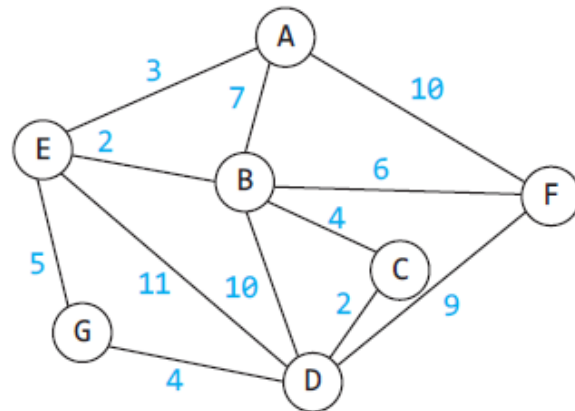
- MST 문제 등



가중치 그래프

```
vertex = [ 'A', 'B', 'C', 'D', 'E', 'F', 'G' ]
weight = [ [ None, 7, None, None, 3, 10, None ],
            [ 7, None, 4, 10, 2, 6, None ],
            [ None, 4, None, 2, None, None, None ],
            [ None, 10, 2, None, 11, 9, 4 ],
            [ 3, 2, None, 11, None, 13, 5 ],
            [ 10, 6, None, 9, 13, None, None ],
            [ None, None, None, 4, 5, None, None ] ]
```

- 최단 경로 문제

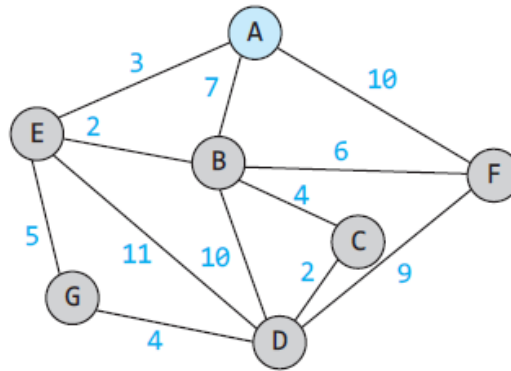


가중치 그래프

```
vertex = [ 'A', 'B', 'C', 'D', 'E', 'F', 'G' ]
weight = [ [ 0, 7, INF, INF, 3, 10, INF ],
            [ 7, 0, 4, 10, 2, 6, INF ],
            [ INF, 4, 0, 2, INF, INF, INF ],
            [ INF, 10, 2, 0, 11, 9, 4 ],
            [ 3, 2, INF, 11, 0, 13, 5 ],
            [ 10, 6, INF, 9, 13, 0, INF ],
            [ INF, INF, INF, 4, 5, INF, 0 ] ]
```

최단거리 문제의 종류

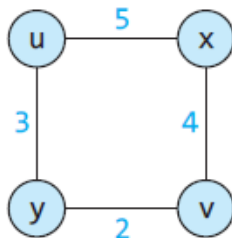
- 문제1) 시작 정점에서 도착 정점까지의 최단 경로 길이
- 문제2) 시작 정점에서 모든 정점까지의 최단 경로 길이



(A→A) 최단 경로 A = 0
 (A→B) 최단 경로 A-E-B = 5
 (A→C) 최단 경로 A-E-B-C = 9
 (A→D) 최단 경로 A-E-B-C-D = 11
 (A→E) 최단 경로 A-E = 3
 (A→F) 최단 경로 A-F = 10
 (A→G) 최단 경로 A-E-G = 8

결과: [0, 5, 9, 11, 3, 10, 8]

- 문제3) 모든 정점 간의 최단 경로 길이



가중치 그래프

$V = ['u', 'v', 'x', 'y']$
 $W = [[0, INF, 5, 3], \quad \# u$
 $[INF, 0, 4, 2], \quad \# v$
 $[5, 4, 0, INF], \quad \# x$
 $[3, 2, INF, 0] \quad \# y$

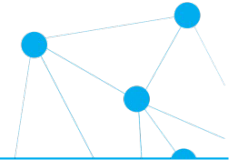
인접 행렬 표현

$D = [[0, 5, 5, 3],$
 $[5, 0, 4, 2],$
 $[5, 4, 0, 6],$
 $[3, 2, 6, 0]]$

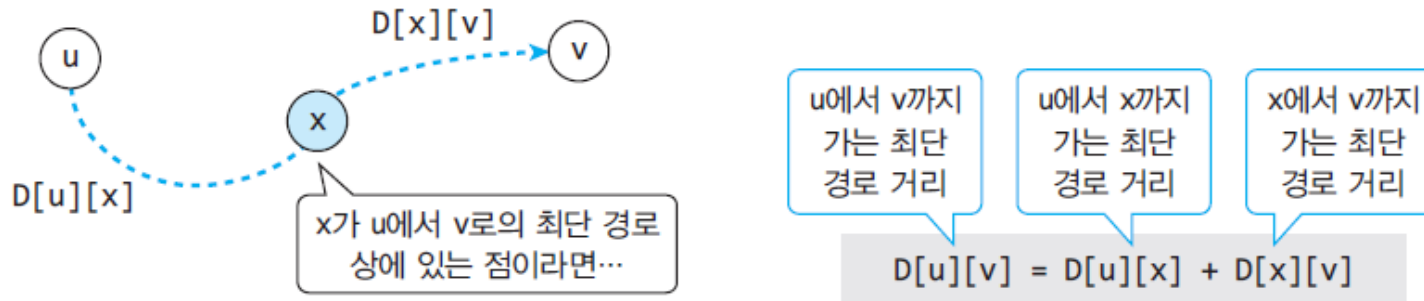
계산 결과

[그림 7.19] 모든 정점에서 모든 정점까지의 최단경로 거리를 구하는 문제와 결과 예

7.6 모든 정점간의 최단 경로 길이



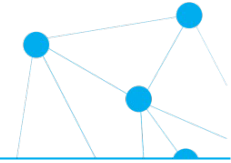
- Floyd-Warshall 알고리즘(Floyd 알고리즘)
 - 동적 계획법 사용
- 최단 경로 문제: 최적 부분 구조 특성을 만족함



[그림 7.20] 최단경로 문제는 최적 부분 구조 특성을 만족함

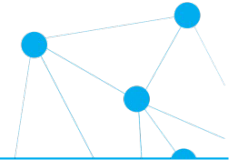
- 최장 경로 문제
 - 최적 부분 구조 특성을 만족하지 않음

순환 관계식



- 인접 행렬 표현 그래프 $W \rightarrow D$
 - 최단거리 행렬 D
 - $D^k[i][j]$:
 - 1~ k 번째 정점까지 만을 이용한 정점 $i \rightarrow j$ 의 최단 경로
 - 최적해: D^n
- 동적 계획 전략
 - $D^0 \rightarrow D^1 \rightarrow D^2 \rightarrow \dots \rightarrow D^n$

순환 관계식



- 기반 상황

- D^0 : 아무런 정점을 거치지 않는 경로 $\rightarrow W$

- 일반 상황

- D^k : k 까지의 정점만을 이용한 부분 문제의 최적해

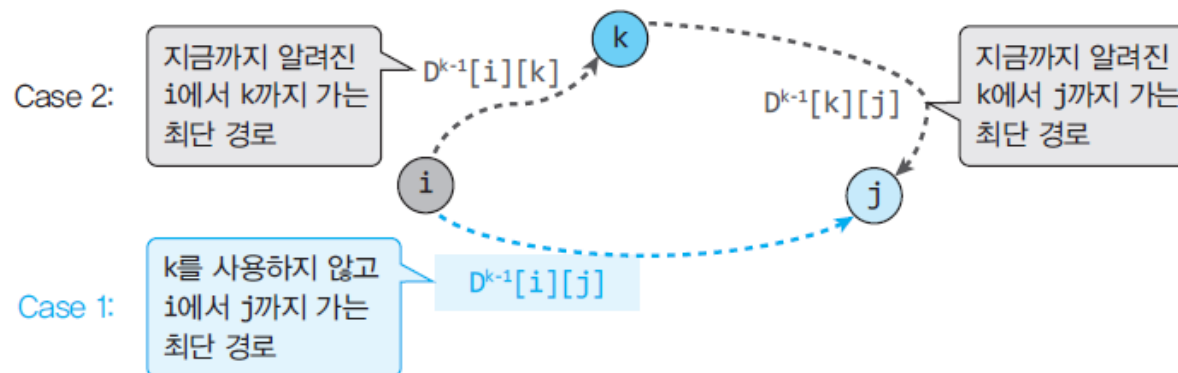
- D^{k-1} 이 구해진 상태에서 k 번째 정점을 고려할때

- 후보1: k 를 거치지 않는 경로

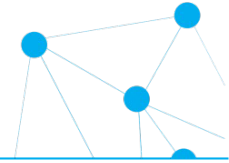
$$D^k[i][j] \leftarrow D^{k-1}[i][j]$$

- 후보2: k 를 거치는 경로

$$D^k[i][j] \leftarrow D^{k-1}[i][k] + D^{k-1}[k][j]$$



알고리즘



- 순환 관계식

$$D^k[i][j] = \begin{cases} W[i][j] & k = 0 \\ \min(D^{k-1}[i][j], D^{k-1}[i][k] + D^{k-1}[k][j]) & \text{otherwise} \end{cases}$$

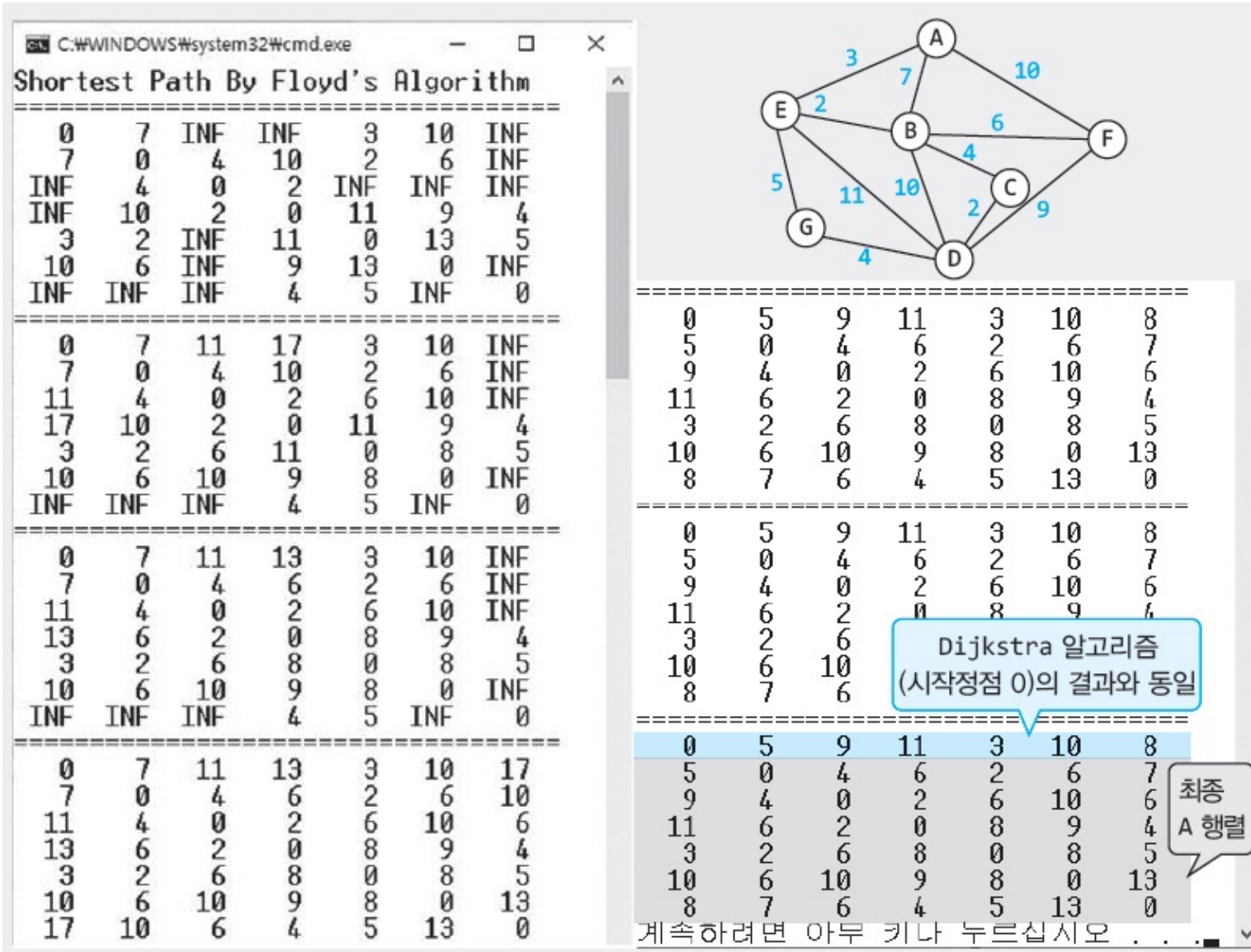
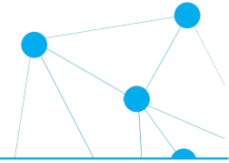
알고리즘 7.10 Floyd의 최단경로 탐색 알고리즘

- 알고리즘
 - 3중 루프

- 복잡도
 - $O(n^3)$

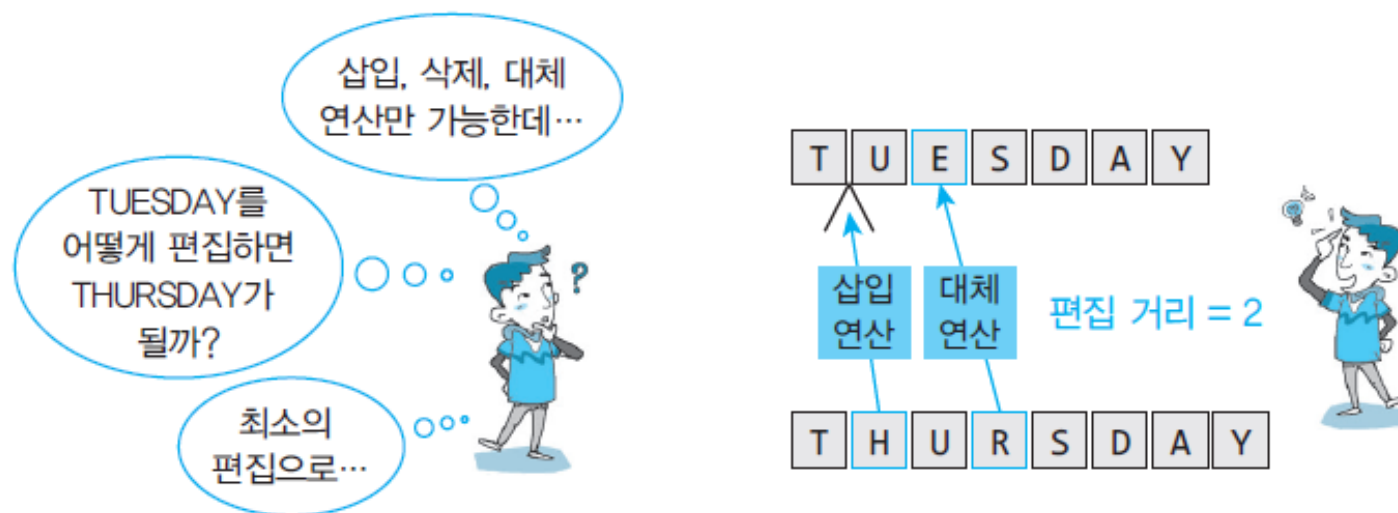
```
01 import copy
02 def shortest_path_floyd(vertex, W):
03     vsize = len(vertex)
04     D = copy.deepcopy(W)
05
06     for k in range(vsize) :
07         for i in range(vsize) :
08             for j in range(vsize) :
09                 if (D[i][k] + D[k][j] < D[i][j]) :
10                     D[i][j] = D[i][k] + D[k][j]
11     printD(D)
```

알고리즘 테스트



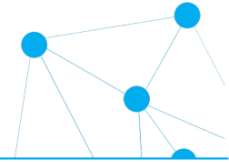
7.7 편집 거리

두 문자열 S 와 T 가 주어졌고 S 를 편집하여 T 로 변환시키려고 한다. 사용할 수 있는 편집 연산은 한 문자의 삽입(insertion)이나 삭제(deletion), 그리고 대체(substitution)연산 뿐이다. S 를 T 로 변환시키는데 필요한 최소 편집 연산의 회수(편집 거리)를 구하라.



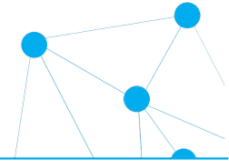
- 입력:
 - 길이가 각각 m 과 n 인 두 문자열 S 과 T
 - $E_{ST}(m, n)$

순환 관계식



- 기반 상황
 - $E_{ST}(m, 0) = m$
 - $E_{ST}(0, n) = n$
- 일반 상황: $S[m], T[n]$
 - Case1: $S[m] = T[n]$
 - $E_{ST}(m, n) = E_{ST}(m - 1, n - 1)$
 - Case2: $S[m] \neq T[n]$
 - 대체 연산: $S[m]$ 을 $T[n]$ 으로 대체
$$E_{ST}(m - 1, n - 1) + 1$$
 - 삭제 연산: $S[m]$ 을 삭제
$$E_{ST}(m - 1, n) + 1$$
 - 삽입 연산: $T[n]$ 을 S 에 삽입
$$E_{ST}(m, n - 1) + 1$$

알고리즘



- 순환식

$$E_{ST}(m, n) = \begin{cases} n & m = 0 \\ m & n = 0 \\ E_{ST}(m-1, n-1) & S[m] = T[n] \\ 1 + \min(E_{ST}(m-1, n-1), E_{ST}(m-1, n), E_{ST}(m, n-1)) & \text{otherwise} \end{cases}$$

- 알고리즘

- 메모이제이션 사용

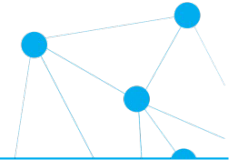
- 복잡도

- $O(mn)$

알고리즘 7.12 편집 거리(동적 계획법, 메모이제이션 사용)

```
01 def edit_distance_mem(S, T, m, n, mem):
02     if m == 0: return n        # S가 공백이면, T의 모든 문자를 S에
03     if n == 0: return m        # T가 공백이면, S의 모든 문자들을 S에
04
05     if mem[m-1][n-1] == None : # 아직 해결되지 않은 문제이면
06         if S[m-1] == T[n-1]:   # S와 T의 마지막 문자가 같으면,
07             mem[m-1][n-1] = edit_distance_mem(S, T, m-1, n-1, mem)
08
09     else: # 그렇지 않으면, 세 연산을 모두 적용해 봄
10         mem[m-1][n-1] = 1 + \
11             min( edit_distance_mem(S, T, m, n-1, mem),
12                 edit_distance_mem(S, T, m-1, n, mem),
13                 edit_distance_mem(S, T, m-1, n-1, mem))
14         # print("mem[%d][%d] = "(m-1,n-1), mem[m-1][n-1])
15
16     return mem[m-1][n-1]      # 해를 반환
```

알고리즘 테스트



알고리즘 테스트 편집 거리(메모이제이션) 테스트

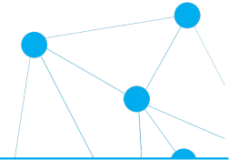
```
S = "tuesday"
T = "thursday"
m = len(S)
n = len(T)
print("문자열: ", S, T)
print("편집거리(분할정복) = ", edit_distance(S, T, m, n))

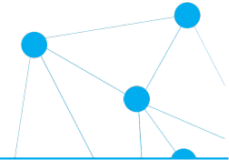
mem = [[None for _ in range(n)] for _ in range(m)]
dist = edit_distance_mem(S, T, m, n, mem)
print("편집거리(메모이제이션) = ", edit_distance_mem(S, T, m, n, mem))
```

```
C:\WINDOWS\system32\cmd.exe
문자열:  tuesday thursday
편집거리(분할정복) = 2
mem[0][0] = 0
mem[1][0] = 1
mem[2][0] = 2
mem[0][1] = 1
mem[1][1] = 1
mem[2][1] = 2
mem[1][2] = 1
mem[2][2] = 2
mem[0][2] = 2
mem[0][3] = 3
mem[1][3] = 2
mem[2][3] = 2
mem[3][4] = 2
mem[4][5] = 2
mem[5][6] = 2
mem[6][7] = 2
편집거리(메모이제이션) = 2
계속하려면 아무 키나 누르십시오 . . .
```

테이블이 채워지는 과정.
edit_distance_mem()
14행의 주석 한 줄을
제거하면 출력됨.

실습 과제





감사합니다!